PROJECT SINGULARITY

The Impossible Website Challenge

One website. Eight brutal modules. Zero shortcuts.

Difficulty: 10 / 10 | **Estimated effort:** 20 engineers x 12 months | **Success rate:** practically zero

1. The Mission

Build **SINGULARITY**: a single website that behaves like a full operating system, a collaborative workspace, a game engine, and a secure messenger at the same time. Everything runs inside the browser. The server is dumb and untrusted; the client does the real work.

Each module on its own is a serious project. The challenge is that all eight must work **together**, inside strict performance and security budgets, with almost no third-party code to lean on.

> **The one-line pitch:** A browser tab where 50 people can co-edit code, compile and run it, view it in a custom 3D engine, and chat about it, all end-to-end encrypted, peer-to-peer, fully offline-capable, accessible to everyone, and unhackable by a red team.

2. The Rules

Rule	Meaning
Zero runtime dependencies	No React, Vue, jQuery, Three.js, Yjs, Socket.IO or any npm library in the shipped bundle. Build tools (bundler, compiler, linter) and the LLVM/Emscripten toolchain are allowed.
Browser-native first	Use Web APIs directly: WebGPU, WebRTC, WebCrypto, WebAssembly, Service Workers, IndexedDB, Web Workers, OPFS.
Untrusted server	The backend must never see plaintext user data, keys, or code. It may only relay ciphertext and discovery metadata.
Single deployable	One public URL. No installs, no plugins, no extensions.
Everything tested	Each module needs automated tests. No test, no points.
Everything explained	Every design decision is written down with trade-offs in an architecture document.

3. The Eight Modules

Each module lists what you must deliver and why it is so difficult.

M1 Real-Time Collaborative Editor (own CRDT)

- Multi-user, conflict-free text editing with live cursors and selections for **50 simultaneous editors**.
- Implement your own CRDT (RGA, Logoot, or Fugue). Yjs and Automerge are not allowed.
- Per-user undo/redo that does not undo other people's work.
- Handle documents of **1,000,000 characters** with no typing lag.
- Offline edits must merge perfectly after reconnecting, with zero lost characters.

Why it is brutal: CRDT correctness is a research-level problem. Interleaving anomalies, tombstone growth, and memory bloat destroy naive implementations.

M2 In-Browser C Compiler and Runtime

- Compile and run C programs **entirely in the browser** using WebAssembly (no server-side compile).
- Terminal emulator with working stdin, stdout, and stderr, written from scratch.
- Run code in a Web Worker with a hard CPU time limit and a memory cap.
- Infinite loops must be killable instantly without freezing the page.
- Show compiler errors inline in the editor from M1 with line and column accuracy.

Why it is brutal: You must ship a toolchain (clang or TCC compiled to WASM) and build a sandboxed syscall layer, a virtual filesystem, and safe preemption on a single-threaded platform.

M3 Custom 3D Engine (no Three.js)

- Renderer written from scratch on **WebGPU**, with a WebGL2 fallback.
- PBR materials, real-time shadow maps, and image-based lighting.
- Draw **100,000 instanced objects at a stable 60 FPS**.
- GPU-computed particle system with at least 1 million particles.
- Hand-written glTF 2.0 loader with skeletal animation.

Why it is brutal: Two graphics APIs, shader code in WGSL and GLSL, and GPU memory management, with no engine to hide behind.

M4 End-to-End Encrypted Layer

- All user data is encrypted client-side with WebCrypto before it leaves the device.
- Implement an X3DH-style key agreement with a Double Ratchet for messages (forward secrecy and post-compromise security).
- Safety-number verification so users can detect a man-in-the-middle attack.
- Encrypted file transfer up to **2 GB** using chunked streaming, with no full-file buffering in RAM.
- Group key management when members join, leave, or get kicked.

Why it is brutal: Cryptography is unforgiving. One misplaced nonce, one reused key, or one timing leak breaks everything. Group rekeying at scale is an open engineering problem.

M5 Peer-to-Peer Mesh Network

- WebRTC data channels with a signaling server used **only** for discovery.
- STUN and TURN fallback for hostile NATs and corporate firewalls.
- Hybrid topology: full mesh for small rooms, relayed tree for large rooms.
- Seamless recovery when peers drop, change network (Wi-Fi to mobile data), or sleep their tab.

Why it is brutal: Real-world networks are chaotic. Symmetric NATs, battery throttling, and flaky mobile links make P2P a constant fight.

M6 Offline-First Sync Engine

- The whole application must work in airplane mode after the first load (Service Worker plus OPFS and IndexedDB).
- Background sync with deterministic conflict resolution across all data types.
- Schema migrations that never lose user data, tested from every previous version.
- Storage quota awareness with graceful eviction and recovery.

Why it is brutal: Offline-first means every feature needs a second implementation path, and every bug can corrupt local data permanently.

M7 Accessibility and Internationalization

- WCAG 2.2 AA across the entire app, **including the canvas and 3D scene** (build a parallel accessible tree).
- Complete keyboard-only navigation and screen-reader support (NVDA, VoiceOver, TalkBack).
- Right-to-left layouts, plus correct text shaping for **Bangla** and **Arabic** inside the code editor.
- Respect prefers-reduced-motion, prefers-contrast, and system font scaling up to 200%.

Why it is brutal: Custom-rendered editors and 3D canvases are invisible to assistive tech by default. You must reinvent the semantics yourself.

M8 Hostile-Environment Hardening

- Strict CSP with no unsafe-inline or unsafe-eval, Trusted Types enforced, SRI on every asset, COOP and COEP enabled.
- Survive automated scans for XSS, CSRF, clickjacking, prototype pollution, SSRF, and request smuggling.
- Rate limiting, abuse detection, and protection against malicious peers inside the P2P mesh.
- Write a complete STRIDE threat model, then survive a **60-minute live red-team session**.

Why it is brutal: You are running arbitrary user-submitted C code (M2) inside a page that holds encryption keys (M4). One sandbox escape means total compromise.

4. Performance and Reliability Budget

These numbers apply to the **whole** system with every module loaded. Missing any one fails the benchmark.

Metric	Target	Test condition
Initial JS payload	170 KB gzip or less	Cold load, modules lazy-loaded
Largest Contentful Paint	1.5 s or less	Throttled 4G, mid-range Android phone
Interaction to Next Paint	100 ms or less	During a 50-user editing session
3D frame rate	60 FPS steady	100k instances plus shadows plus 1M particles
Lighthouse scores	95 or higher in all four categories	Mobile profile
Memory after 1 hour	400 MB or less, no leaks	Heap snapshots compared at 10 min and 60 min
Concurrent users	10,000 with p95 latency under 200 ms	Distributed load test
Chaos survival	Kill any server node, lose zero data	Random failure injection for 30 min

5. Scoring and Tiers

Total: **1000 points**. Each module is worth up to 100 points, the budget is 100 points, and integration is 100 points.

Tier	Requirement	Points	Realism
Bronze	Any 2 modules working independently	up to 200	A strong student can do this
Silver	4 modules plus basic integration	up to 450	A skilled small team
Gold	6 modules, budget mostly met, M8 passed	up to 750	A professional team over months
Platinum	All 8 modules integrated, budget met	up to 950	Elite team, near a year
IMPOSSIBLE	Platinum plus survive red-team plus chaos test with zero failures	1000	Almost nobody

6. Deliverables

- **Source repository** with a clean commit history and a README that builds the project in one command.
- **Live deployment** on a public URL with valid HTTPS.
- **Architecture document** covering module boundaries, data flow, and every major trade-off.
- **Threat model** (STRIDE) plus a written security test report.
- **Automated test suite**: unit, integration, end-to-end, fuzzing for the parser and crypto layers.
- **Load and chaos test report** with graphs and raw data.
- **Demo video** (10 minutes) showing all modules working together in one session.

7. Survival Guide: Suggested Architecture

Layer	Suggested approach
Main thread	Only UI and input. Everything heavy moves to workers. Keep the main thread free of long tasks.
Worker pool	Separate workers for CRDT merge, WASM runtime, crypto, and sync, talking over typed messages and SharedArrayBuffer.
Rendering	OffscreenCanvas in a dedicated worker for 3D, so editing never stalls the frame loop.
Storage	OPFS for large binary data, IndexedDB for structured data, append-only logs for crash safety.
Networking	WebRTC mesh for rooms, WebSocket only for signaling, with a ciphertext-only relay fallback.
Crypto	WebCrypto only. Never invent your own primitives. Use constant-time comparisons and unique nonces.
Testing	Deterministic simulation: record and replay network schedules to reproduce CRDT and sync bugs.

8. Recommended Build Order

- **Phase 1:** M6 (offline storage) and M8 basics (CSP, headers). Everything else sits on top of these.
- **Phase 2:** M1 (CRDT editor). Get convergence proven with property-based tests before adding networking.
- **Phase 3:** M5 and M4 (P2P and encryption) so the editor becomes truly collaborative and private.
- **Phase 4:** M2 (C compiler in WASM), the biggest single technical risk. Prototype it early.
- **Phase 5:** M3 (3D engine) and M7 (accessibility). Plan accessibility from day one, not as an afterthought.
- **Phase 6:** Integration, performance tuning, chaos testing, and the red-team session.

9. Honest Reality Check

> **This is intentionally almost impossible.** Companies like Figma, Google, and Signal have each spent years and large teams on pieces of this. Finishing all eight modules alone is not the real goal. The real goal is to push your limits and learn the deepest parts of web engineering, security, and systems design.
>
> **For a solo student:** aim for the **Bronze or Silver tier**. A strong pairing is **M4 (encryption) plus M8 (hardening)**, which fits a cybersecurity path, or **M1 plus M5** for a real-time collaboration project you can show in a portfolio. A small module done perfectly, with tests and documentation, is worth more than eight modules done badly.

Challenge accepted? Pick your first module, write your threat model first, and start building.